Super-Resolution of Satellite AOD over Dallas–Fort Worth

Year-split results and the updated RQ3 (uncertainty)

Ryan Seidel and Rency Ajit Kansagra · CSE 6367 Computer Vision · October 30, 2026

Contents

1. Part 1 · Year-split results (RQ1, RQ2, RQ3)
2. Part 2 · RQ3 in full: uncertainty and the guaranteed range
3. Part 3 · RQ3 note: why uncertainty is hard to quantify (June 2023 record)
4. Part 4 · RQ3 plan (to-do list)

Train 2018–2021 · validation 2022 · test 2023 (never used for training or choosing anything).

PART 1 · YEAR-SPLIT RESULTS (RQ1, RQ2, RQ3)

Year-split results: RQ1, RQ2 and RQ3

What this is in one sentence

We train all three SR models on Rency's real split by year (**2018–21 train / 2022 val / 2023 test**), with and without the average-back rule, and score them **once** on 2023, a year they never saw.

These numbers replace the preliminary June 2023 numbers in `sprint3_rq1_README.md`. Those only checked that the code works.

Summary

RQ	Answer on 2023
RQ1 (beat bicubic?)	**Yes.** All 6 models, 8–15% lower RMSE. Best: SwinIR-light, RMSE 0.0107 vs 0.0126, +1.4 dB PSNR.
RQ2 (average-back rule)	**Works at no cost.** The output re-averages to the input almost exactly (8.7e-09), with the same or better accuracy and detail.
RQ3(a) (does $\hat{\sigma}$ point at the hard pixels?)	**Yes.** Real error rises 5–7× from the most- to the least-confident pixels, and $\hat{\sigma}$ rises with it.
RQ3(b) (guaranteed 90% range?)	**Yes.** After one factor fitted on 2022, every model's range contains ≥ 90% of 2023 pixels (6.5–7.2% outside).
RQ4 (EPA PM2.5)	Not yet on the year split: needs Rency's daily maps with coordinates.

How we updated the research questions

RQ1, RQ2 and RQ4 are unchanged. **RQ3 was rewritten** after the first year-split results and the instructor's feedback ("the uncertainty may be hard to quantify").

	Old (proposal)	New
RQ3	Is the confidence map reliable? When it reports 80% sure, is it correct about 80% of the time?	**(a)** Does the confidence map point at the pixels where the model is actually wrong? **(b)** Can we turn it into a range that is guaranteed to contain the true value for at least 90% of pixels in an unseen year, and is that range better than one that ignores the confidence map?
How it's tested	Coverage at one level (e.g. 90%)	(a) Error vs $\hat{\sigma}$ groups, 4×4 block positions, calibration curve, V-RMSE. (b) Conformal calibration on 2022, check on 2023, compared with a constant-width range

Why we changed it:

1. **The old test can't tell uncertainty apart from an average.** A range of the **same width everywhere** can hit 90% coverage without knowing anything about which pixels are hard. Passing the old RQ3 therefore didn't show that $\hat{\sigma}$ is real uncertainty. Part (a) tests exactly that.
2. **The old test had no clear "done".** Raw coverage moved between 0.79 and 0.97 depending on the model and the epoch. Part (b) gives a yes/no answer with a statistical guarantee.
3. **Both parts come from published methods** (papers in `uncertainity_pdfs/`):

    - Zheng et al. 2026, *Self-Supervised Uncertainty Estimation for Super-Resolution of Satellite Images*: our loss makes σ^2 estimate the expected squared error at each pixel. Their checks (V-RMSE, coverage, sharpness, sub-grid analysis) are what part (a) uses.
    - Belhasin et al., *PUQ* (TPAMI 2024), and Angelopoulos et al., *im2im-uq* (ICML 2022): conformal risk control, i.e. calibrate the range so that misses are $\leq \alpha$ with probability $\geq 1 - \delta$. That's part (b).
4. We kept **four RQs** (RQ3 has two parts) rather than adding an RQ5, so the proposal's structure stays the same.

The full RQ3 write-up is in `rq3_uncertainty_results.md`, and in the second part of this PDF.

How to run it

```
python -W ignore -m src.train --data years --model edsr   --epochs 200 --eval_every 10
python -W ignore -m src.train --data years --model unet   --epochs 200 --eval_every 10
python -W ignore -m src.train --data years --model swinir --epochs 200 --eval_every 10
```

Add `--consistency` to each for the average-back rule. That's 6 runs in total.

- Every `--eval_every` epochs it scores **2022 val**, and it remembers the epoch with the lowest val RMSE.
- At the end it goes back to that epoch and scores **2023 test once** (the `TEST 2023` line).
- 2023 is never used to choose anything, so the test score is fair.
- Models are saved to `checkpoints/<model>[_consistency]_years.pt`.

How long it takes (RTX 3080)

Model	200 epochs
EDSR	~8 min
U-Net	~7 min
SwinIR-light	~50 min

The data (from Rency, `data/split_data/`)

Set	Years	Patches	What it's for
Train	2018–2021	4,543	The model learns from these
Val	2022	1,142	Picks the best epoch
Test	2023	976	Scored once, at the end

- Same format as the Sprint 2 file: `lr` 16×16, `hr` 64×64, `mask`, `scale = 4`. Values are already real AOD.
- Normalization uses Rency's **training-only** mean and std (`train_stats.npz`: 0.1035, 0.0958).
- Different years never share pixels, so no overlap check is needed (unlike the June split).
- Loaded by `make_year_datasets()` in `src/dataset.py`.

Results (2023 test year, 200 epochs, best epoch picked on 2022)

	Parameters	RMSE ↓	vs bicubic	PSNR ↑	SSIM ↑	Detail (1.0 = right)	Consistency ↓	Coverage90 (target 0.90)	Cal. error ↓	Mean $\hat{\sigma}$	Best epoch
Bicubic	–	0.0126	–	37.97	0.927	0.49	–	–	–	–	–
EDSR	1.52M	0.0116	−8%	38.71	0.940	0.64	1.5e-03	**0.91**	**0.033**	0.0102	80
EDSR + rule	1.52M	0.0109	−13%	39.24	0.948	0.71	8.7e-09	0.81	0.066	0.0073	140
U-Net	1.93M	0.0110	−13%	39.14	0.948	0.72	1.7e-03	0.83	0.052	0.0073	70
U-Net + rule	1.93M	0.0110	−13%	39.21	0.949	0.73	8.7e-09	0.79	0.081	0.0068	40
SwinIR-light	0.92M	**0.0107**	**−15%**	**39.42**	**0.951**	0.74	6.4e-04	0.79	0.083	0.0067	110
SwinIR-light + rule	0.92M	**0.0107**	**−15%**	39.40	**0.951**	**0.75**	8.7e-09	0.79	0.086	0.0067	110

"Rule" = the average-back rule (`--consistency`). Full logs are in `runs/years_<model>.log`.

RQ1: can deep SR beat simple interpolation?

Yes. All six models beat bicubic on 2023 on every score: 8–15% lower RMSE, +0.7 to +1.4 dB PSNR, +0.013 to +0.024 SSIM.

- **SwinIR-light is the best**, with the fewest parameters.
- The models add real detail. Detail goes from 0.49 (bicubic) to 0.64–0.75 while the error drops, so the new detail is real, not made up. They're still smoother than the real map (1.0).
- Every best epoch is well before 200 (40–140), so 200 epochs is enough.
- The gain is about 3× larger than on the June split (−15% vs −5%). That's because there's about 40× more training data.

RQ2: does the average-back rule reduce made-up detail without smoothing the map?

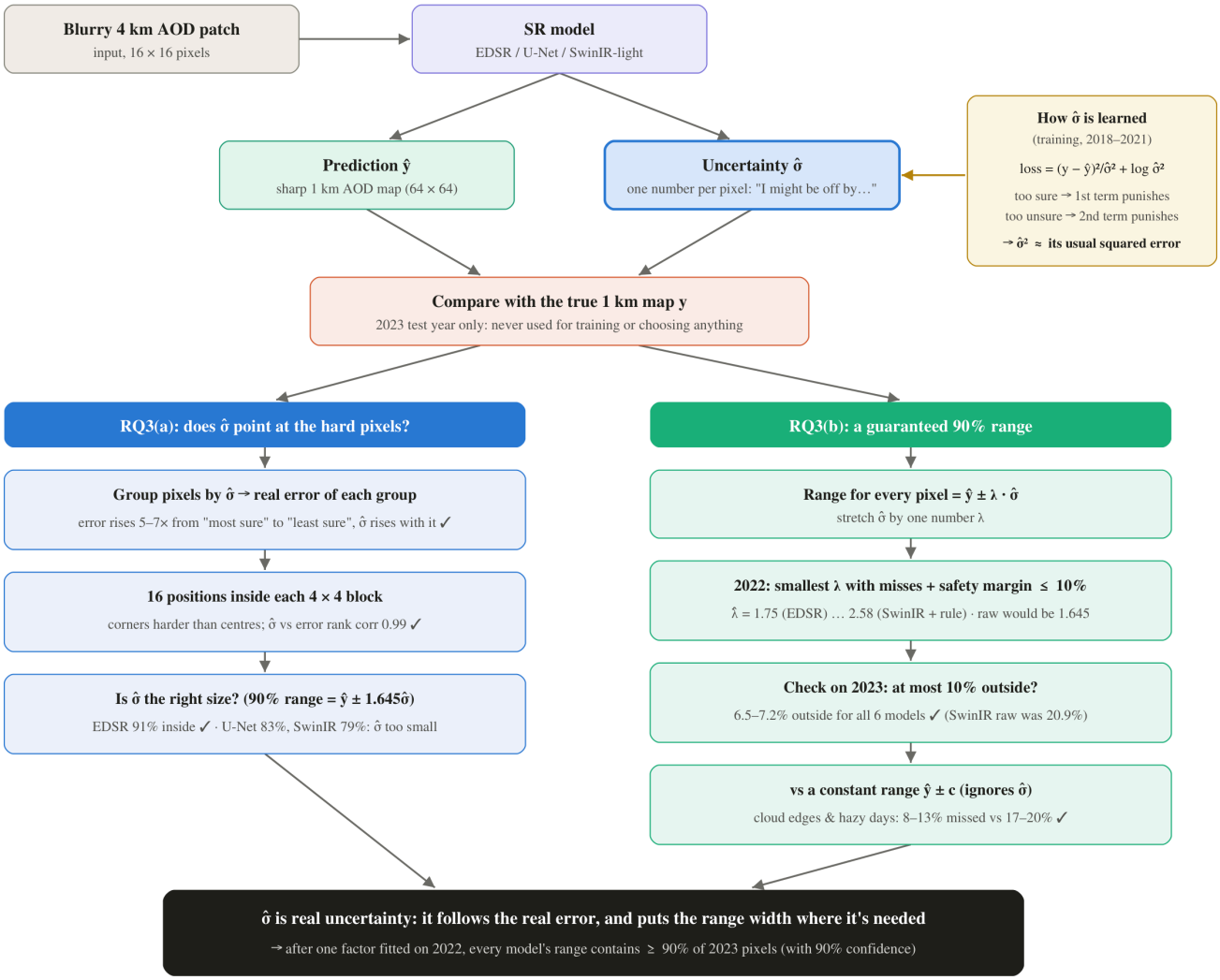
It keeps the output faithful to the input at no cost.

- Consistency error drops from 6e-04–2e-03 to **8.7e-09**: the sharp map now averages back to the blurry input almost exactly. (The rule is a hard correction applied after the network, so this is by construction.)
- Accuracy is equal or better: EDSR improves (−8% → −13%), and U-Net and SwinIR stay the same.
- Detail goes up slightly, not down (e.g. EDSR 0.64 → 0.71). So the rule does **not** smooth the map back toward bicubic.

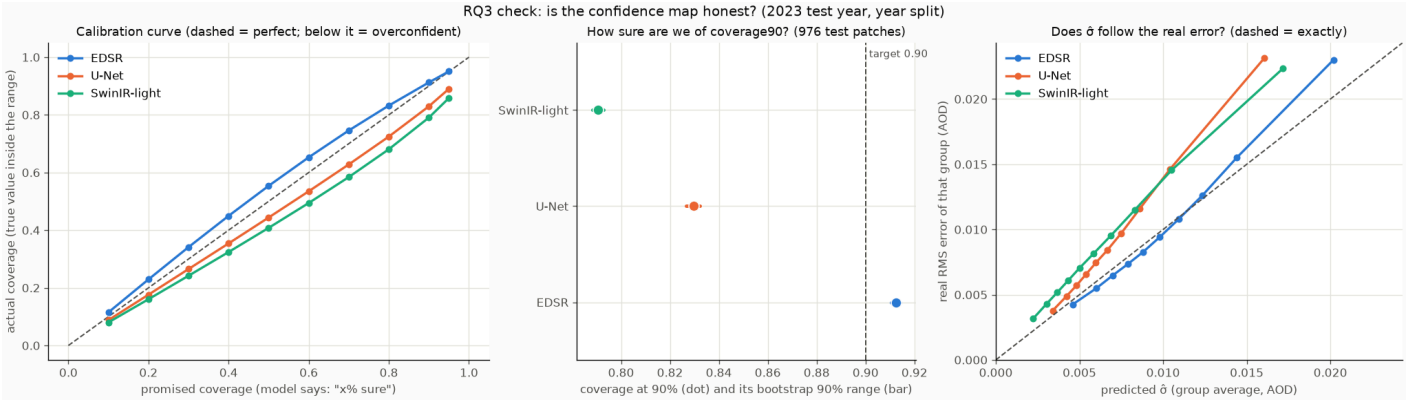
RQ3: is the confidence map real uncertainty?

First look (raw $\hat{\sigma}$, the old RQ3 test): only plain EDSR is honest. The more accurate models are **overconfident**: when they say "90% sure", the true value is inside the range only 79–83% of the time. The sections below explain what that means.

With the new two-part RQ3 (full details in `rq3_uncertainty_results.md`):



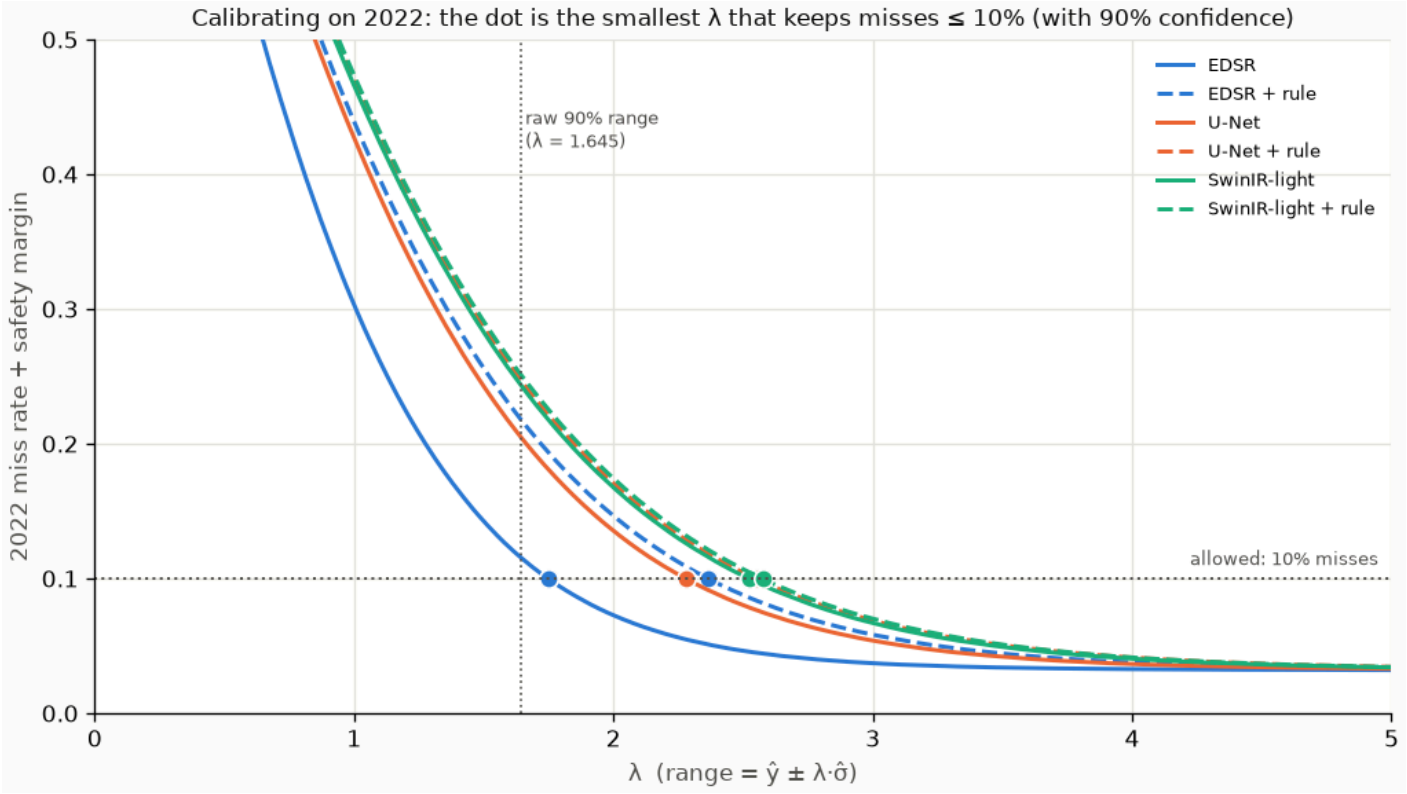
(a) Does σ̂ point at the hard pixels? Yes.



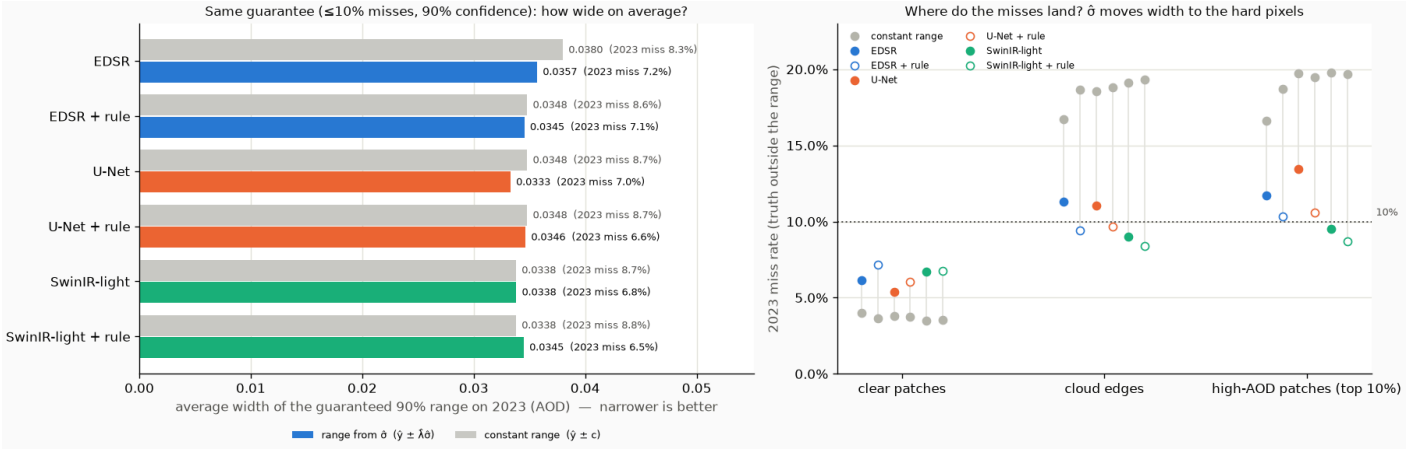
- Left: calibration curve. EDSR sits on the diagonal; U-Net and SwinIR fall below it (overconfident).
- Right: pixels grouped by σ̂. The real error rises 5–7× from the most- to the least-confident group, and σ̂ rises with it. EDSR is on the diagonal (the right size); U-Net and SwinIR are correctly ranked but 20–30% too small.

- Across the 16 positions inside each 4×4 block, $\hat{\sigma}$ and the error have a rank correlation of 0.99: block corners are harder than block centres, and $\hat{\sigma}$ learned that on its own.

(b) A guaranteed 90% range? Yes, for every model. Range = $\hat{y} \pm \lambda \cdot \hat{\sigma}$, with λ fitted on 2022.



Model	$\hat{\lambda}$ (raw = 1.645)	Raw range: 2023 misses	Guaranteed range: 2023 misses
EDSR	1.75	8.8%	**7.2%**
EDSR + rule	2.37	18.8%	**7.1%**
U-Net	2.28	17.1%	**7.0%**
U-Net + rule	2.55	20.8%	**6.6%**
SwinIR-light	2.53	20.9%	**6.8%**
SwinIR-light + rule	2.58	21.3%	**6.5%**



- Left: on average, the $\hat{\sigma}$ range is about as wide as a constant-width range with the same guarantee.

- Right: the difference is **where the misses land**. The constant range misses **17–20%** of pixels at cloud edges and on hazy days. The $\hat{\sigma}$ range keeps those at **8–13%**, because it's narrow on easy pixels and wider on hard ones. That's the evidence that $\hat{\sigma}$ is real uncertainty, not an average error.

Caveat

Each model was trained **once, with one seed**. Small gaps (U-Net vs SwinIR, rule vs no rule for SwinIR) could be run-to-run noise. The gap to bicubic is consistent across all six runs, so that part is solid.

How the confidence map works

1. The model outputs two maps, not one

For each 64×64 patch, the network outputs two channels:

Output	What it is
$\hat{\mathbf{y}}$ (`sharp`)	the sharpened 1 km AOD: bicubic + the detail the network adds
log $\hat{\sigma}^2$ (`log_var`)	how **unsure** the model is at each pixel

In code (`src/models/sr_net.py`), the last layer of EDSR, U-Net or SwinIR is split into two heads: one for the detail, one for the uncertainty. The model predicts the **log** of the variance, not the variance itself, because:

- a variance can't be negative, but a log can be any number, so no special handling is needed;
- it's clamped to [−10, 10] so `exp()` can't blow up.

To get the uncertainty in AOD units: $\hat{\sigma}$ = exp(0.5 · log_var) × std, where std is the training normalization (0.0958).

2. How it learns to be unsure: the loss

There is **no "true uncertainty" map to copy**. Nobody tells the model how unsure it should be. It learns this from the loss (`uncertainty_loss` , a Gaussian negative log-likelihood, Kendall & Gal 2017):

```
loss per pixel = 0.5 × [ (y − ŷ)² / σ²   +   log σ² ]
                         error term          cost of saying "unsure"
```

- **Error term:** being wrong is divided by $\hat{\sigma}^2$. If the model says "I'm unsure here" (big $\hat{\sigma}$), mistakes at that pixel cost less.
- **Cost term:** saying "unsure" isn't free. A big $\hat{\sigma}$ makes `log σ²` bigger.
- The best the model can do at each pixel is set **$\hat{\sigma}^2$ ≈ the squared error it usually makes there**:
  - say "unsure" too much → the cost term punishes it;
  - say "sure" too much → the error term punishes it.
- So $\hat{\sigma}$ learns to be big where the model tends to be wrong (cloud edges, sharp plumes) and small where it's easy (flat, clear areas).
- Cloud pixels are masked out, like in every other loss.

This is also why the training loss can go **negative**: `log σ²` is negative when $\hat{\sigma}$ is small.

Is $\hat{\sigma}$ uncertainty, or just the loss?

The key difference: **the loss needs the true answer, and $\hat{\sigma}$ doesn't.**

	Loss / error	Uncertainty $\hat{\sigma}$
Needs the true 1 km map?	**Yes**: $(y - \hat{y})^2$ can't be computed without y	**No**: the network outputs it from the blurry input alone
When does it exist?	After you know the answer	Before; it's a **prediction**
What is it?	How wrong the model *was*	How wrong the model *expects to be* at this pixel
Used when deployed?	No, there's no truth for real 4 km data	Yes

- **Analogy:** a forecast of "70% chance of rain" is made without knowing tomorrow's weather. Checking whether it rained is the scoring. $\hat{\sigma}$ is the forecast; the error maps and red pixels in the figures are only the scoring.
- The loss is used in **training** to teach $\hat{\sigma}$ which kinds of pixels tend to be wrong. On 2023 the model writes down $\hat{\sigma}$ first; the truth is used only afterwards, to check it.
- **Evidence it's a prediction, not an average loss:** $\hat{\sigma}$ varies about 10× within 2023 (0.002 to 0.02 AOD). It ranks the error before seeing it (5–7× more error in the "least sure" group). It learned structure nobody told it about (block corners, cloud edges, plumes). And a constant-width range, which is what "the average loss" would give, fails at cloud edges and on hazy days, where the $\hat{\sigma}$ range doesn't.
- **What kind of uncertainty:** aleatoric (how much the true 1 km value can vary given this blurry input), as defined by Kendall & Gal 2017 and Zheng et al. 2026. It doesn't cover "a day unlike anything in training" (that needs an ensemble) or MAIAC's own error.

3. How we check it: coverage

If $\hat{\sigma}$ is honest and errors are roughly bell-shaped (Gaussian), then:

The model says	The range is	The true value should fall inside
"90% sure"	$\hat{y} \pm 1.645\ \hat{\sigma}$	90% of the time
"80% sure"	$\hat{y} \pm 1.282\ \hat{\sigma}$	80% of the time
"50% sure"	$\hat{y} \pm 0.674\ \hat{\sigma}$	50% of the time

We count how often that actually happens on 2023 (`src/metrics.py`):

Number	Meaning	Good is
coverage90	fraction of real pixels where $\lvert y - \hat{y} \rvert \le 1.645\ \hat{\sigma}$	0.90
cal_err	the same check at 10 levels (10%…95%), averaging how far off each one is	0
mean $\hat{\sigma}$ (`unc`)	the average uncertainty the model reports	small, **as long as** coverage is right

- Coverage **below** the target = **overconfident** (ranges too narrow).
- Coverage **above** the target = **underconfident** (ranges too wide).
- Mean $\hat{\sigma}$ is there because a model could get perfect coverage by saying "very unsure" everywhere. That would be useless.

4. What the results say

A quick rule of thumb: an honest $\hat{\sigma}$ should be about the same size as the model's typical error (RMSE).

Model	RMSE	Mean $\hat{\sigma}$	$\hat{\sigma}$ / RMSE	Coverage90
EDSR	0.0116	0.0102	0.88	0.91 ✅
U-Net	0.0110	0.0073	0.66	0.83
SwinIR	0.0107	0.0067	0.63	0.79

- **EDSR's $\hat{\sigma}$ is about the size of its error**, so its ranges are about right.
- **U-Net and SwinIR shrink $\hat{\sigma}$ much more than they shrink their error.** Their ranges are about a third too narrow, so they're overconfident.
- **The average-back rule makes this worse**, e.g. EDSR 0.91 → 0.81. The rule shifts $\hat{y}$ after the network, but $\hat{\sigma}$ isn't adjusted for that shift. Also, the more accurate a model gets on training years, the more confident it becomes, and 2023 is a little harder than the training years.
- The ordering (calibrated but less accurate vs accurate but overconfident) is a useful finding for the report: **the best-scoring model is not the most trustworthy one.**

5. Seeing it pixel by pixel (images)

```
python -W ignore -m src.rq3_uncertainty_maps --model edsr     # -> docs/diagrams/rq3_maps_edsr.png
python -W ignore -m src.rq3_uncertainty_maps --model swinir   # -> docs/diagrams/rq3_maps_swinir.png
```

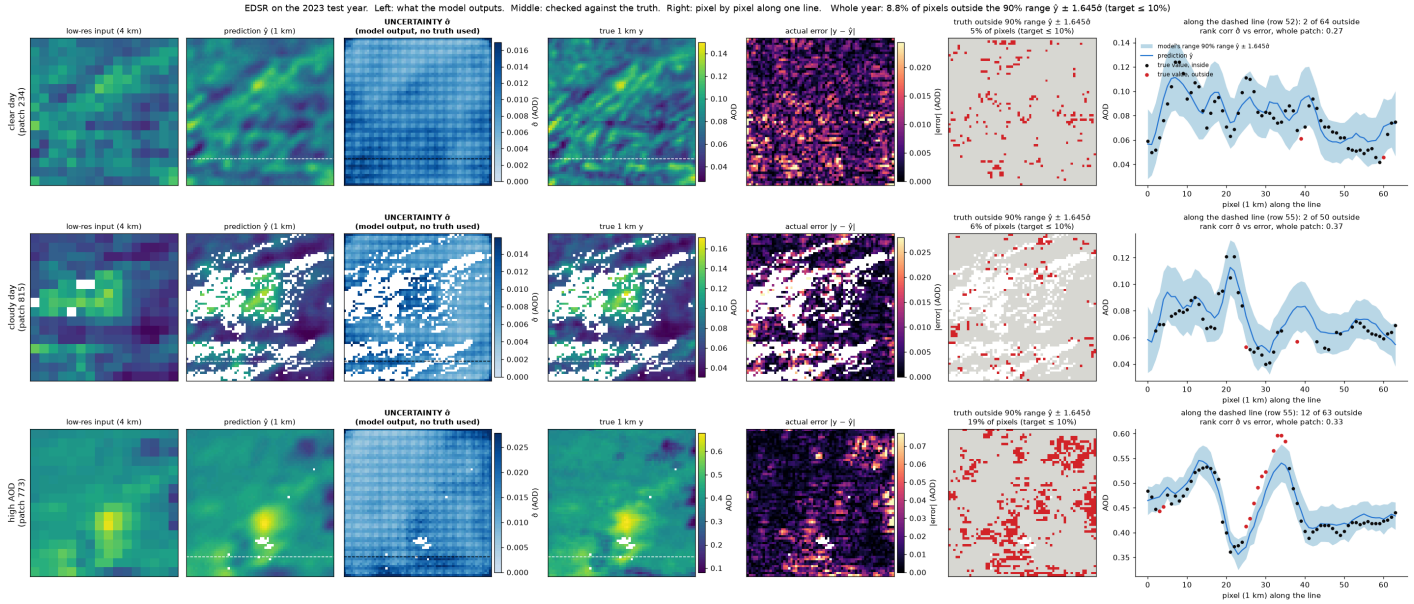
Add `--consistency` for the models trained with the rule, or `--patches 12 340` to pick your own 2023 patches.

Each row is one 2023 test patch, read left to right in three parts:

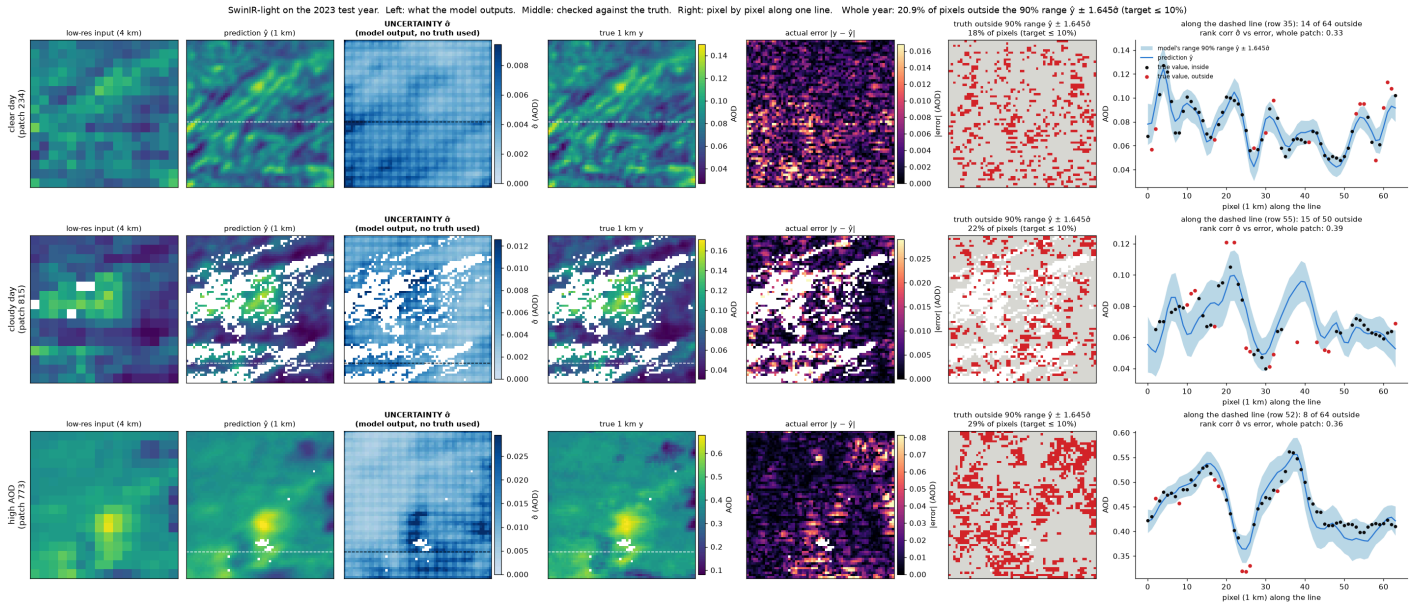
Part	Panels	What it shows
1. What the model outputs	low-res input · prediction $\hat{y}$ · **uncertainty $\hat{\sigma}$** (blue)	The two maps the model makes from the blurry input alone. **No truth is used.** Dark blue = "I'm unsure here".
2. Checked against the truth	true 1 km · actual error · truth outside the 90% range (red)	Where the model was really wrong, and the pixels where the true value fell outside $\hat{y} \pm 1.645\hat{\sigma}$. Honest ≈ 10% red.
3. Pixel by pixel along one line	line plot	One row of the patch (the dashed line). Blue line = prediction, blue band = the model's 90% range **for each pixel**, dots = true values (red = outside the band).

The line plot is the clearest demonstration: **every pixel gets its own error bar**, and the band is wider where the model is less sure.

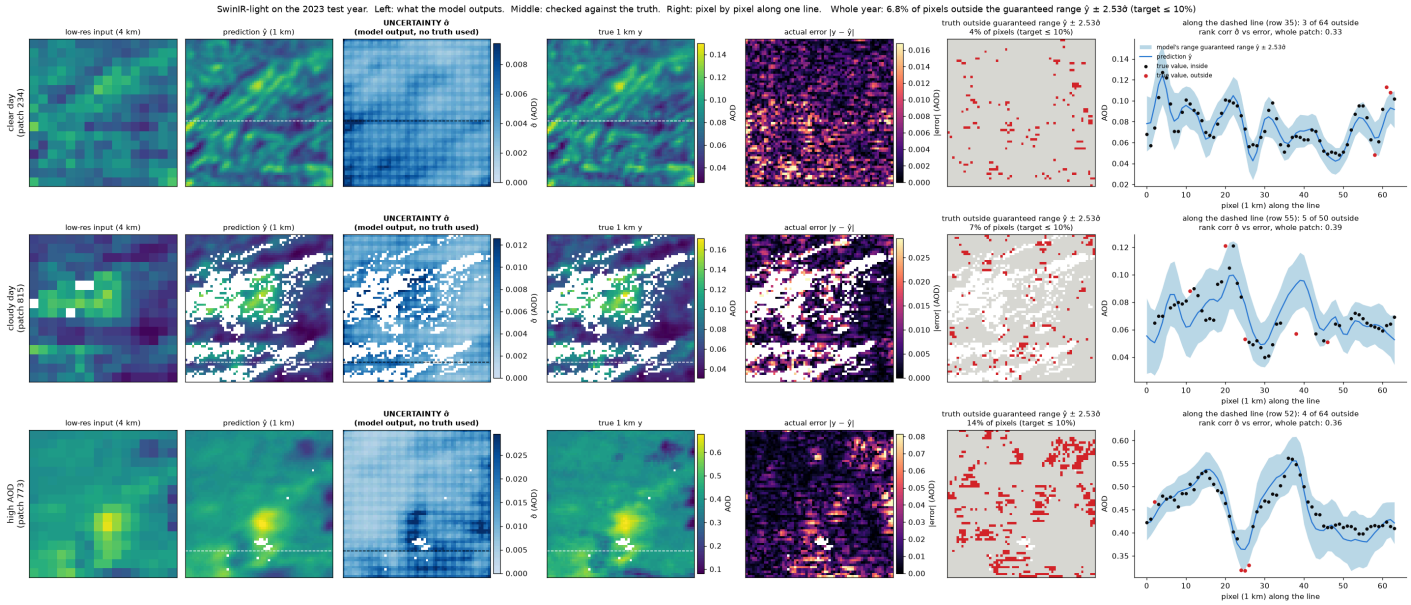
EDSR, raw 90% range ($\hat{y} \pm 1.645\hat{\sigma}$):



SwinIR-light, raw 90% range (ŷ ± 1.645σ̂):



SwinIR-light, guaranteed range (ŷ ± 2.53σ̂, calibrated on 2022): same σ̂ map; only the band is wider. Whole year: 20.9% → 6.8% outside.



Patch	EDSR outside 90% range	SwinIR outside 90% range
Clear day (234)	5%	18%
Cloudy day (815)	6%	22%
High AOD (773)	**19%**	**29%**
Whole 2023 year	**8.8%** ✅	**20.9%**

What the pictures show:

- **σ̂ does point at the hard pixels.** On the cloudy day, σ̂ is darkest along cloud edges, which is also where the error is biggest. On the hazy day it's darkest around the plume, and in the line plot the band widens over the AOD peak.
- **But not wide enough on the hazy day:** in the EDSR line plot, the true peak (≈ 0.60) sits above the whole band (≈ 0.54), a run of red dots in a row. The model knew the peak was uncertain, but underestimated by how much.
- **σ̂ is always high along the patch border**, where the model has no neighbours on one side. The line plot skips the 8 border rows for that reason.
- **EDSR is honest on normal days:** red pixels are few and scattered.
- **SwinIR's red pixels are spread everywhere**, so its ranges are too narrow **across the board**. That's the kind of error one rescale factor (fitted on 2022) can fix.
- **Both are overconfident on the high-AOD patch**, and the red pixels **cluster** where the error is biggest (top right). That can't be fixed with one factor. It's the "never seen a day like this" uncertainty from point 6 below.
- σ̂ shows a faint **4-pixel grid**, matching the 4×4 blocks of the low-res input: the model is less sure about pixels far from block centres.
- The rank correlation between σ̂ and |error| within a patch is only **0.27–0.39**. The map points the right way, but roughly.

6. What the confidence map does NOT measure

σ̂ only says **how hard it is to recover MAIAC's 1 km value from its 4 km average**. It does not include:

1. MAIAC's own retrieval error (MAIAC 1 km AOD is itself an estimate);
2. "I've never seen a day like this" uncertainty, e.g. a smoke plume in a new season. One network can't capture that; an ensemble can;
3. any uncertainty about PM2.5.

More detail and the earlier June checks are in `rq3_uncertainty_note.md`.

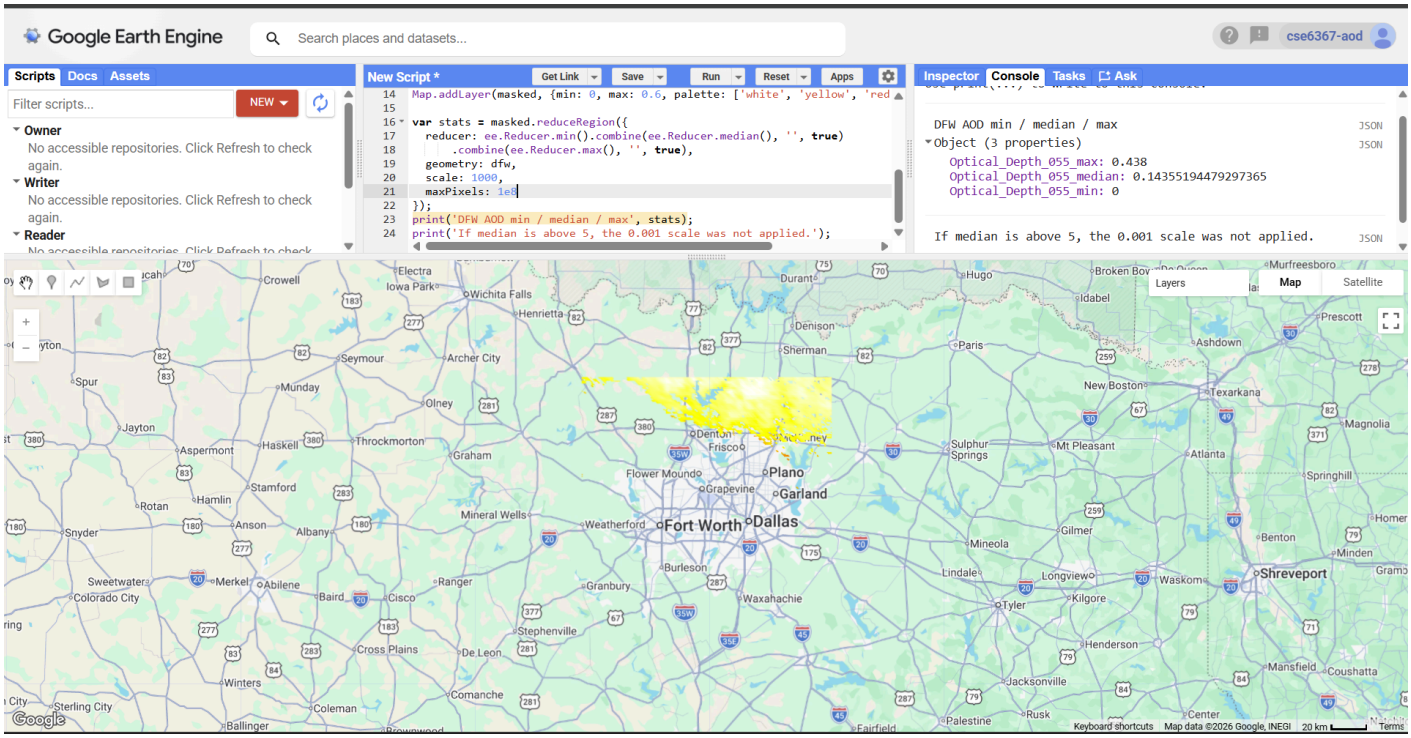
What's next

- ☑ **RQ3 fix:** a guaranteed range, calibrated on 2022 (`rq3_uncertainty_results.md`, part b). Every model ≤ 7.2% misses on 2023.
- ☑ **RQ3 figures on 2023:** calibration curve, error vs $\hat{\sigma}$, 4×4 positions, and the per-patch images (`rq3_uncertainty_results.md`, part a).
- ☐ **Seeds:** retrain the two best models with 2–3 seeds, to see whether the small gaps are real.
- ☐ **RQ4:** rerun the EPA comparison on 2023. This needs Rency's daily maps with coordinates.
- ☐ **Ask Rency:** add `date` and patch `(row, col)` to the patch files, for per-day error bars and monitor matching.
- ☐ **main.tex:** training is on the DFW year patches (not the whole tile), the data is standardized (not log-transformed), and RQ3 uses the new two-part wording (agree with Rency first).

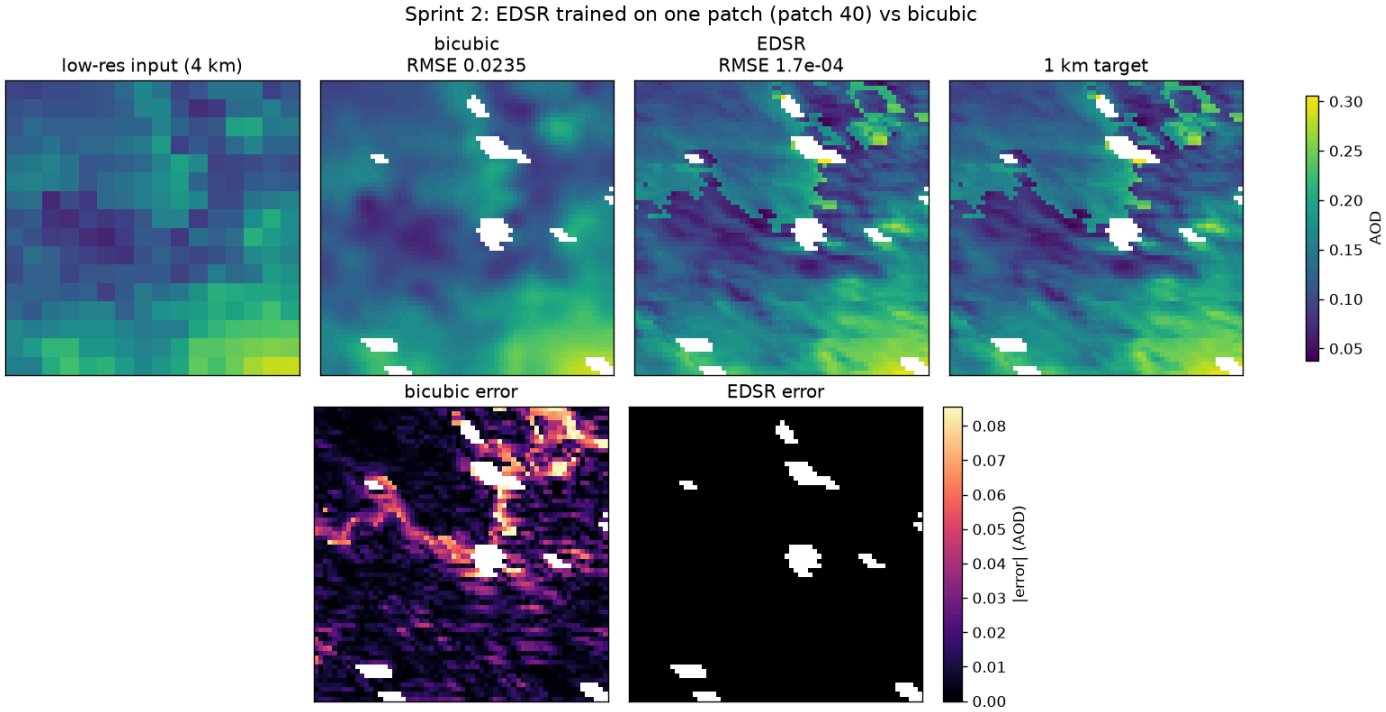
Appendix: earlier figures (before the year split)

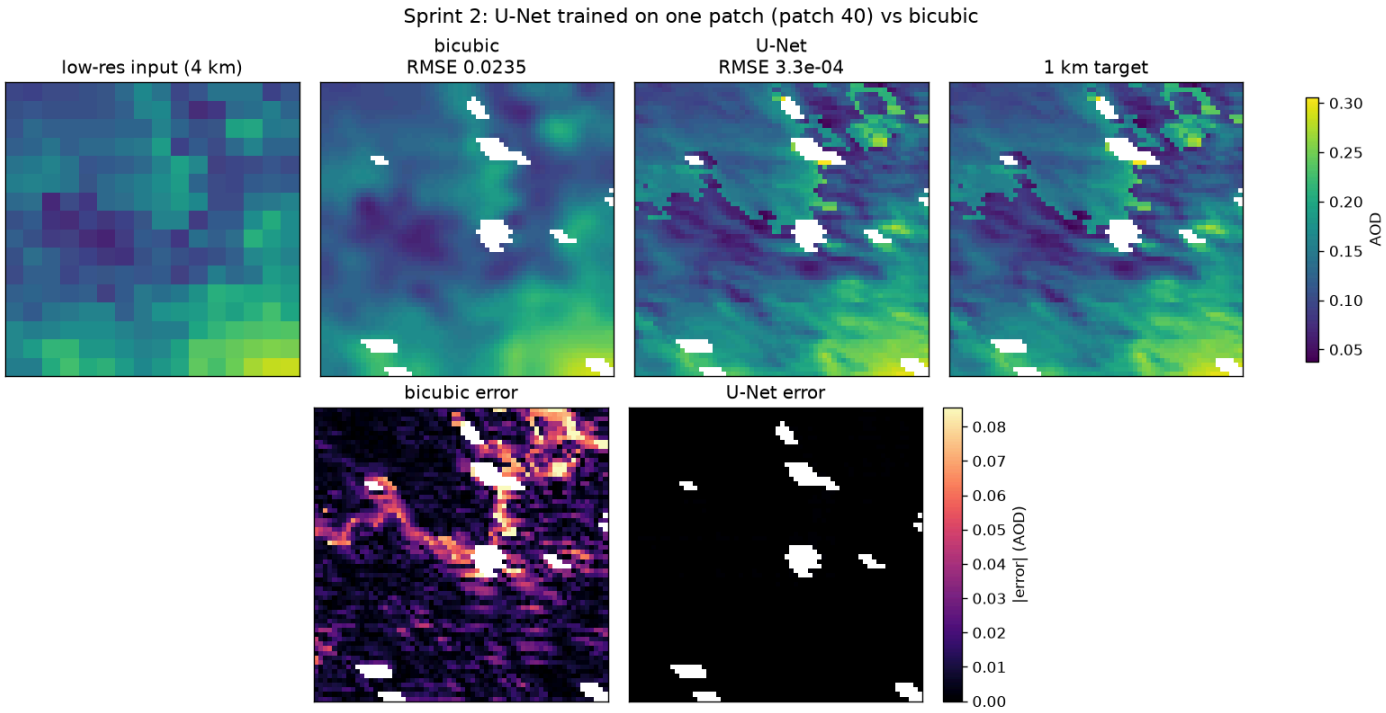
Kept for the record. These come from the Sprint 1–3 work on the June 2023 data.

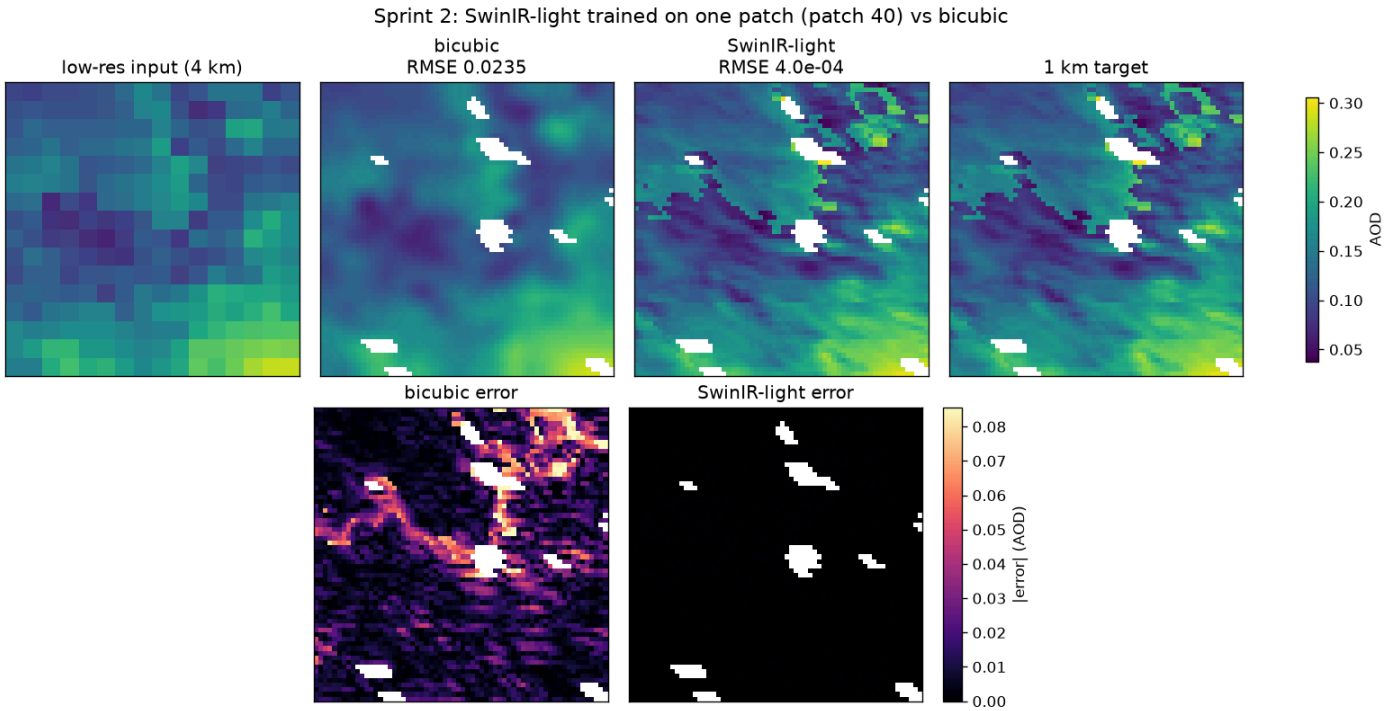
Sprint 1: first DFW MAIAC AOD map.



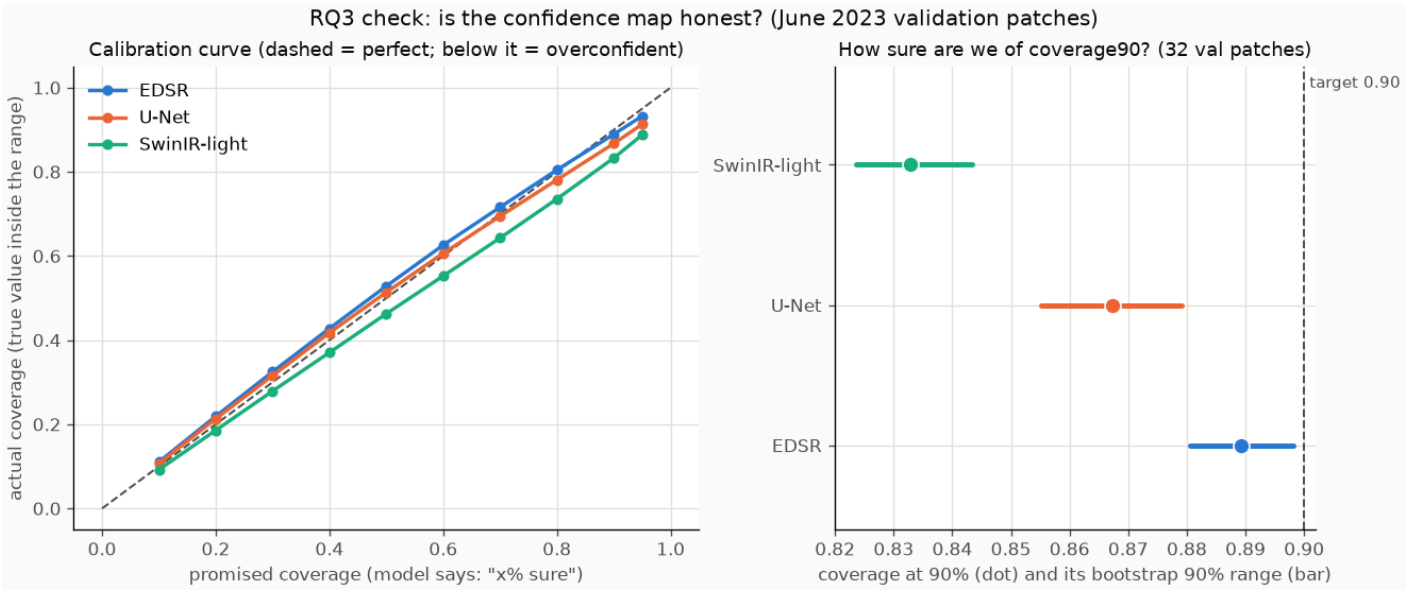
Sprint 2: the "learn one patch" test. All three models pass (error < 10% of bicubic's).



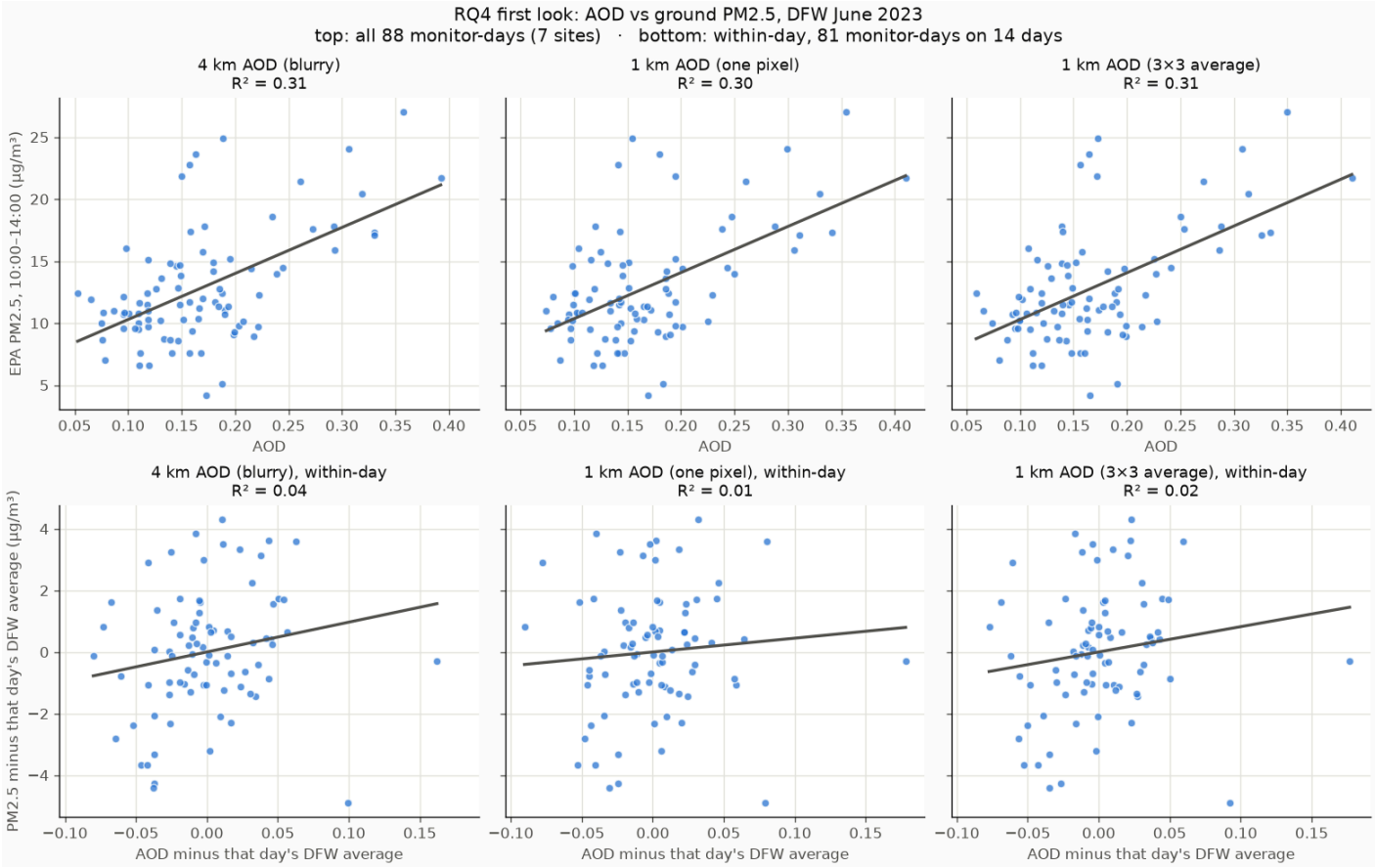




First RQ3 check (June 2023, 3 validation days). Superseded by the 2023 figures above.



First RQ4 look (June 2023): AOD vs EPA PM2.5. $R^2 \approx 0.3$ at both 4 km and 1 km; not enough data to separate them.


RQ4 first look: AOD vs ground PM2.5, DFW June 2023
top: all 88 monitor-days (7 sites) · bottom: within-day, 81 monitor-days on 14 days
4 km AOD (blurry)
R² = 0.31
1 km AOD (one pixel)
R² = 0.30
1 km AOD (3×3 average)
R² = 0.31
EPA PM2.5, 10:00–14:00 (µg/m³)
AOD
4 km AOD (blurry), within-day
R² = 0.04
1 km AOD (one pixel), within-day
R² = 0.01
1 km AOD (3×3 average), within-day
R² = 0.02
PM2.5 minus that day's DFW average (µg/m³)
AOD minus that day's DFW average

PART 2 · RQ3 IN FULL: UNCERTAINTY AND THE GUARANTEED RANGE

RQ3: is the confidence map real uncertainty, and can we guarantee it?

The question (new wording, replaces the old RQ3)

> **RQ3:** (a) Does the confidence map point at the pixels where the model is actually wrong? (b) Can we turn it into a range that is guaranteed to contain the true value for at least 90% of pixels in an unseen year, and is that range better than one that ignores the confidence map?

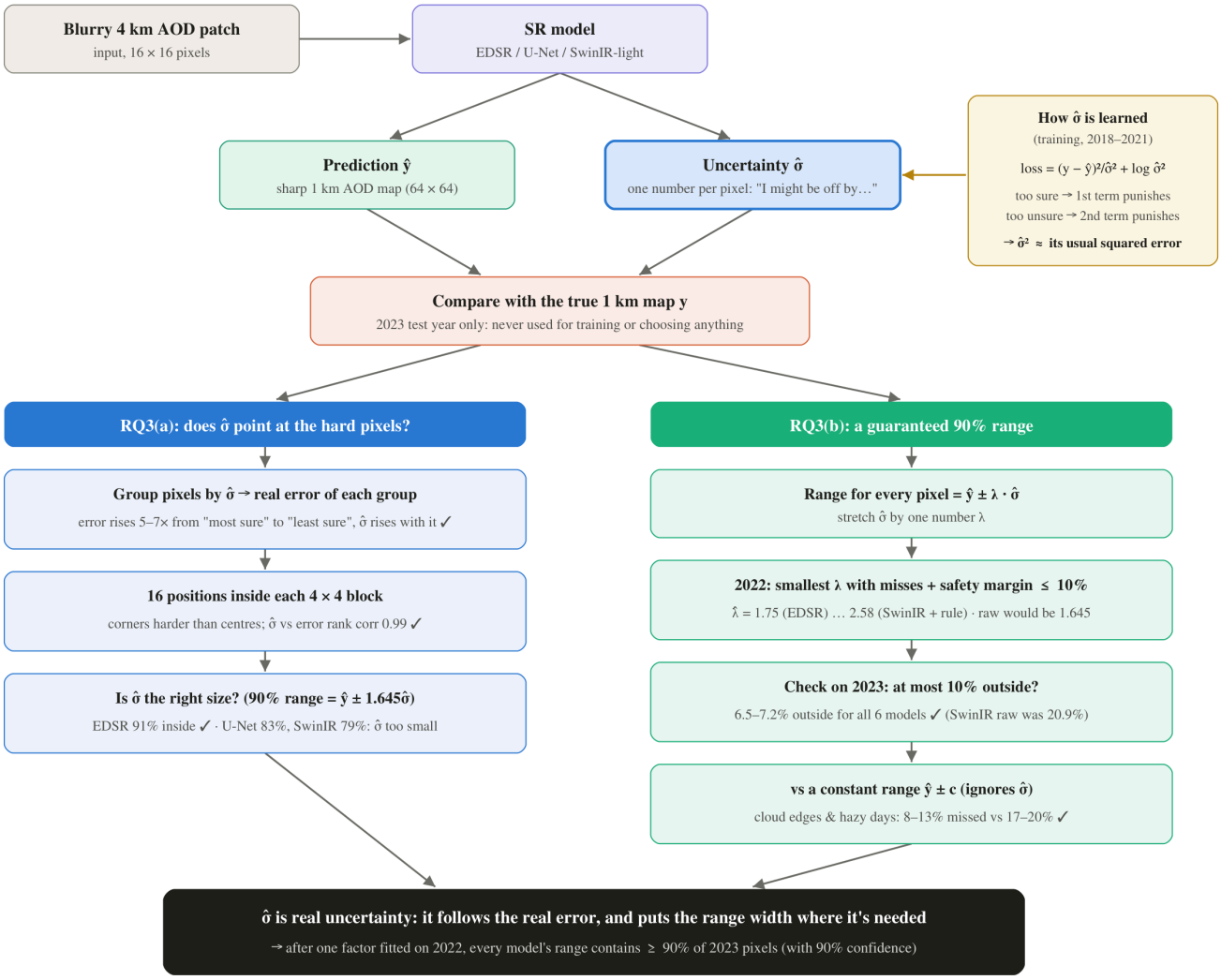
Why the old RQ3 wasn't enough. The old question was "when it says 80% sure, is it right 80% of the time?" A range that is the **same width everywhere** can pass that test without knowing anything about which pixels are hard. So passing it doesn't prove there is uncertainty in the map. (a) tests that $\hat{\sigma}$ knows *which* pixels are hard, and (b) gives a definite, guaranteed answer.

Based on two papers (copies in `uncertainity_pdfs/`, see `papers/references.md`):

- **Zheng et al. 2026**: our loss makes σ^2 learn the **expected squared error at each pixel**, whatever the shape of the error distribution. Their checks: V-RMSE, calibration curve, sharpness, and per-sub-grid analysis. → RQ3(a)
- **Belhasin et al. (PUQ) / Angelopoulos et al. (im2im-uq)**: **conformal risk control**. Stretch the range by one factor λ, chosen on calibration data, so that the miss rate is $\leq \alpha$ with a statistical guarantee. → RQ3(b)

All numbers are on the **2023 test year**, which is never used for training or for choosing anything.

How the uncertainty is measured (flow chart)



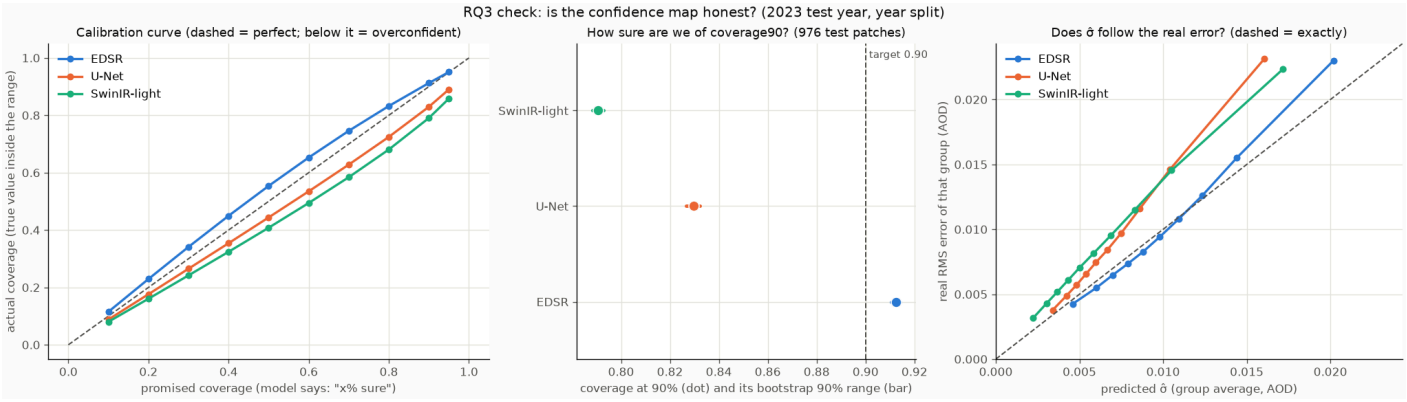
(Editable source: `diagrams/rq3_flowchart.svg`. The PNG is for slides.)

In words:

1. The model makes **two maps** from the blurry input alone: the prediction ŷ and the uncertainty σ̂. It never sees the true 1 km map at this point.
2. σ̂ was learned in training through the loss, which forces it toward the size of the error the model usually makes at that kind of pixel.
3. **RQ3(a)** compares σ̂ with the real error on 2023, pixel by pixel.
4. **RQ3(b)** stretches σ̂ by one factor λ, chosen on 2022, and checks on 2023 that the guarantee holds.

RQ3(a): does σ̂ point at the pixels where the model is wrong?

```
python -W ignore -m src.rq3_uncertainty_check --data years
```



1. Group the pixels by σ̂, then look at their real error

All 3.66 million real 2023 pixels, split into 10 equal groups from "most sure" to "least sure":

σ̂ group	EDSR σ̂ → real RMS error	U-Net σ̂ → real error	SwinIR σ̂ → real error
most sure 10%	0.0046 → 0.0043	0.0034 → 0.0038	0.0022 → 0.0032
middle (5th)	0.0088 → 0.0083	0.0060 → 0.0074	0.0050 → 0.0071
least sure 10%	0.0202 → 0.0230	0.0161 → 0.0232	0.0172 → 0.0223

- **The real error rises about 5–7× from the "most sure" to the "least sure" group, and σ̂ rises with it** for all three models (right panel). σ̂ knows which pixels are hard.
- **EDSR's σ̂ is about the right size** (on the diagonal). U-Net and SwinIR are ranked correctly but **too small by about 20–30%** (above the diagonal). That's a scale problem, which part (b) fixes.

2. The 16 positions inside each 4×4 block

Each low-res pixel covers a 4×4 block of 1 km pixels. Average error and average σ̂ for each of the 16 positions (EDSR):

	RMS error	mean σ̂
block corners	0.0132–0.0139	0.0116–0.0121
block centre (4 pixels)	0.0081–0.0089	0.0077–0.0080

Rank correlation between σ̂ and error across the 16 positions: **0.99 for all three models.** Pixels near the centre of a block are easier to recover; pixels at the corners sit between neighbouring low-res values and are harder. σ̂ learned this on its own. That's the faint grid in the σ̂ maps, and it's real, not an artifact (Zheng et al. found the same thing with a 2×2 grid).

3. Summary numbers

Model	Coverage at 90%	Calibration error	V-RMSE ↓	Mean 90% width	Rank corr σ̂ vs |error|	Cloud edges / open sky coverage
EDSR	**0.91**	**0.033**	3.98e-4	0.0335	0.41	0.87 / 0.92
U-Net	0.83	0.052	3.95e-4	0.0240	0.44	0.77 / 0.83
SwinIR-light	0.79	0.083	**3.74e-4**	**0.0220**	**0.47**	0.77 / 0.79

- Per pixel the rank correlation is only 0.41–0.47, because a single pixel's error is very noisy. Averaged over groups (tables above) the link is very strong.
- SwinIR has the **best-ranked** $\hat{\sigma}$ (highest correlation, lowest V-RMSE) but the **worst-scaled** (79% coverage). EDSR is the reverse.

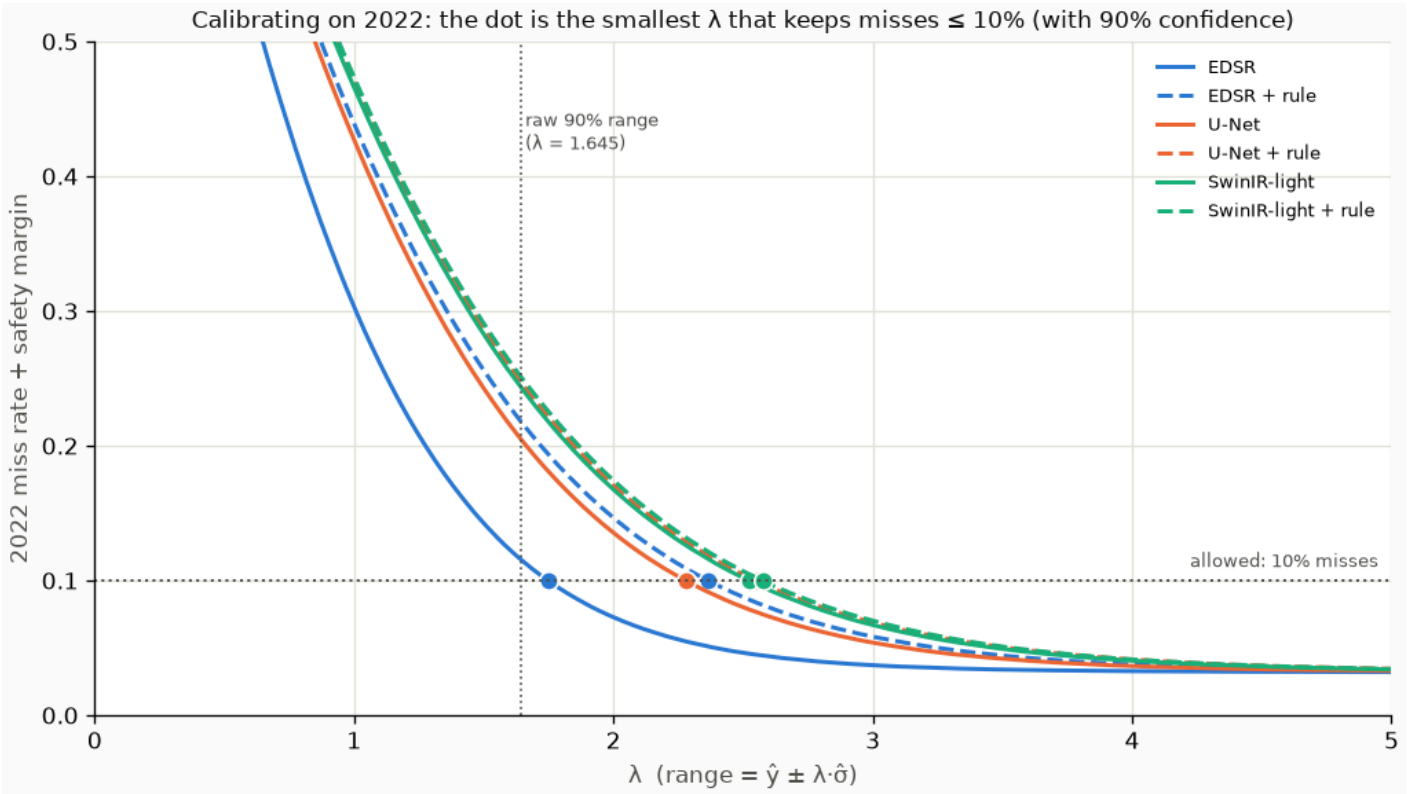
Answer to (a): yes. $\hat{\sigma}$ points at the pixels where the model is wrong: by $\hat{\sigma}$ group, by position in the block, and at cloud edges and plumes (images below). It's informative, but on two of the three models it's too small.

RQ3(b): a range with a guarantee

```
python -W ignore -m src.rq3_guaranteed_range
```

How it works

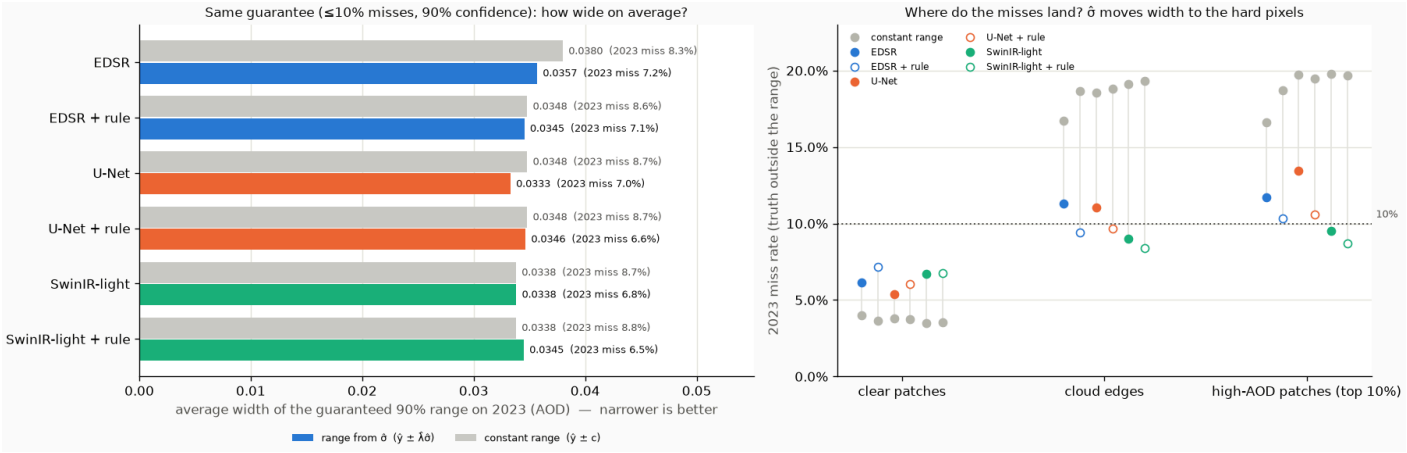
1. Range for every pixel: $\hat{y} \pm \lambda \cdot \hat{\sigma}$. (The raw "90% range" is $\lambda = 1.645$.)
2. On **2022** (1,142 patches): for each λ, the miss rate = fraction of real pixels outside the range, averaged over patches.
3. Add a **safety margin** for only having 1,142 calibration patches (Hoeffding: $\sqrt{(\ln(1/\delta)/2n)} = 0.032$).
4. Pick the **smallest λ with miss rate + margin ≤ 10%**. Then, with 90% confidence ($\delta = 0.10$), the miss rate on new data is ≤ 10%.
5. Check on **2023**.
6. Do the same for a **constant range $\hat{y} \pm c$** that ignores $\hat{\sigma}$, as the baseline.



Results (2023)

Model	$\hat{\lambda}$ (raw = 1.645)	Raw 90% range: 2023 misses	**Guaranteed range: 2023 misses**	Mean width (AOD)	Constant range: misses / width
EDSR	1.75	8.8%	**7.2%** ✅	0.0357	8.3% / 0.0380
EDSR + rule	2.37	18.8%	**7.1%** ✅	0.0345	8.6% / 0.0348
U-Net	2.28	17.1%	**7.0%** ✅	0.0333	8.7% / 0.0348
U-Net + rule	2.55	20.8%	**6.6%** ✅	0.0346	8.7% / 0.0348
SwinIR-light	2.53	20.9%	**6.8%** ✅	0.0338	8.7% / 0.0338
SwinIR-light + rule	2.58	21.3%	**6.5%** ✅	0.0345	8.8% / 0.0338

- **The guarantee holds for every model on 2023 (6.5–7.2% misses, limit 10%).** Models that were overconfident (up to 21% misses) are fixed by one number $\hat{\lambda}$.
- $\hat{\lambda}$ tells you how overconfident each raw $\hat{\sigma}$ was: EDSR needs only 6% wider (1.75 vs 1.645); SwinIR needs 54% wider.
- **Average width is about the same as the constant range** (0.033–0.036 AOD, about ±0.017). So $\hat{\sigma}$ does **not** make the range narrower on average. Its value is in where the width goes:



Where the misses land (2023)

	$\hat{\sigma}$ range: misses	**constant range: misses**
clear patches	5.4–7.2%	3.5–4.0%
cloud edges	**8.4–11.3%**	**16.7–19.3%**
high-AOD patches (top 10%)	**8.7–13.4%**	**16.6–19.8%**

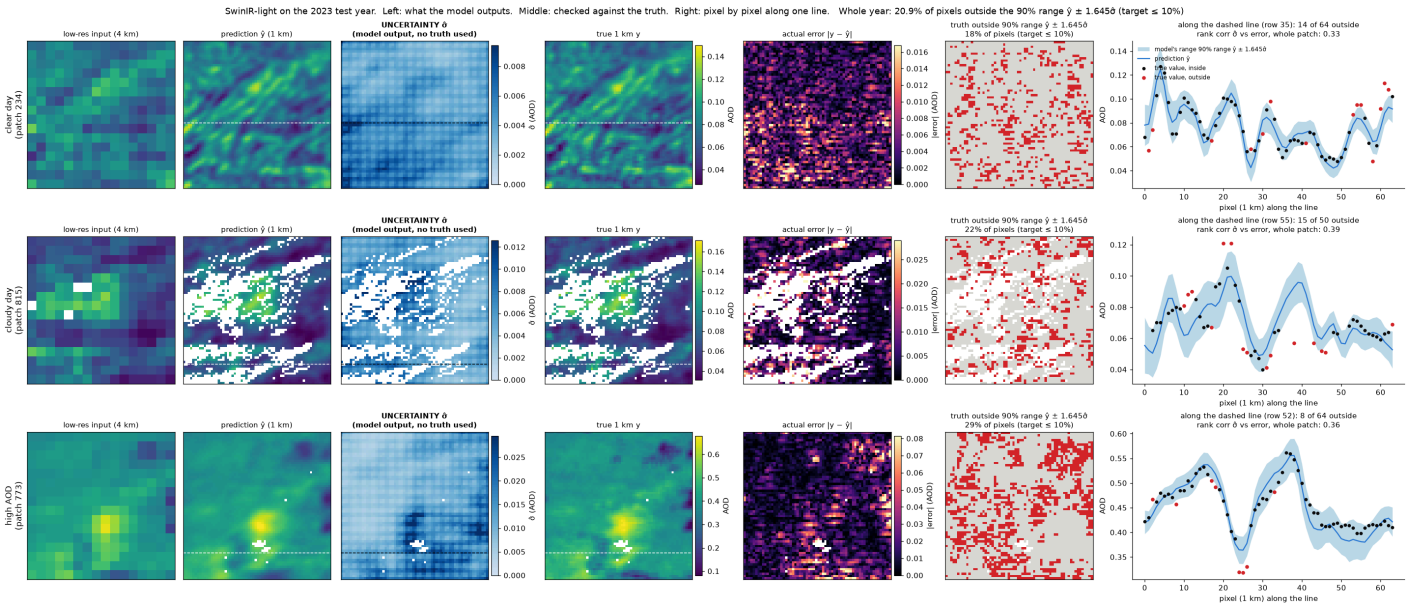
- The constant range meets the 10% target **by over-covering easy clear pixels and failing on the hard ones**: about 1 in 5 pixels missed at cloud edges and on hazy days, exactly where a health user would need it.
- The $\hat{\sigma}$ range is **narrow on easy pixels and up to 2× wider on hard ones** (e.g. SwinIR: 0.024 on clear patches, 0.051 at cloud edges). The misses end up spread much more evenly. SwinIR with the rule stays under 10% in every group.

Answer to (b): yes, with a guarantee. After one calibration factor, every model's range contains ≥ 90% of 2023 pixels. The range built from $\hat{\sigma}$ isn't narrower on average than a constant one, but it's **far more reliable where it matters** (cloud edges, hazy days). That's the practical evidence that $\hat{\sigma}$ carries real uncertainty.

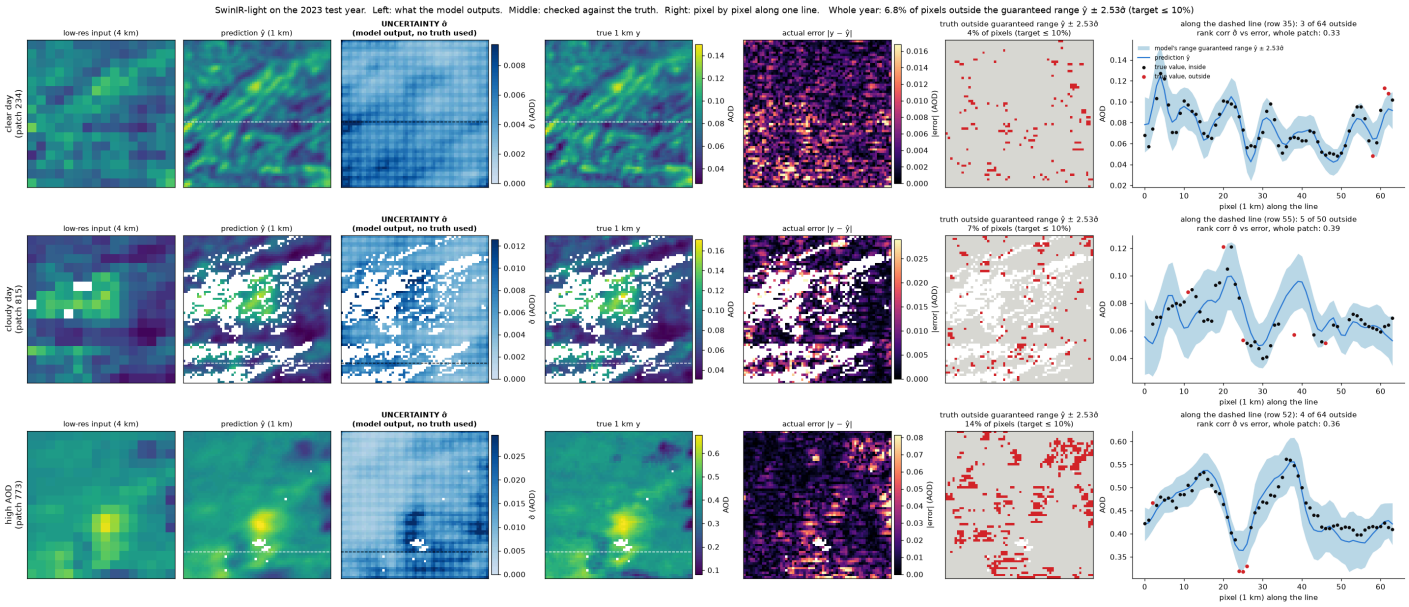
Images: raw range vs guaranteed range

```
python -W ignore -m src.rq3_uncertainty_maps --model swinir                 # raw 90% range (1.645 σ)
python -W ignore -m src.rq3_uncertainty_maps --model swinir --calibrated    # guaranteed range (λ̂ σ)
```

Raw 90% range (ŷ ± 1.645σ): 20.9% outside



Guaranteed range (ŷ ± 2.53σ̂): 6.8% outside



Same $\hat{\sigma}$ map; only the band in the line plot gets wider. With the guaranteed range the red dots mostly disappear, except on the hazy patch (14% outside in that one patch). The guarantee is on the average over all patches, not

on every single patch.

Limits (say these in the report)

- **The guarantee is an average over patches**, not per patch or per pixel. One hazy patch can still miss 14%.
- **It assumes 2022 and 2023 behave alike.** Patches from the same day are correlated, so the real number of independent samples is smaller than 1,142, and the safety margin is a bit optimistic. Fixing this needs a `date` in each patch file, so calibration can be done by day (ask Rency).
- **$\hat{\sigma}$ is uncertainty about MAIAC's own 1 km value.** It doesn't include MAIAC's retrieval error versus the ground, or uncertainty about PM2.5.
- One training run (one seed) per model.

Files

File	What it does
`src/rq3_uncertainty_check.py --data years`	RQ3(a): calibration curve, error vs $\hat{\sigma}$ groups, 4×4 positions, V-RMSE, sharpness
`src/rq3_guaranteed_range.py`	RQ3(b): calibrate λ on 2022, check on 2023, constant-range baseline; saves `runs/rq3b_lambda.json`
`src/rq3_uncertainty_maps.py [--calibrated]`	The per-patch images and line plots
`src/metrics.py`	`v_rmse`, `sharpness` (new), `coverage`, `calibration_error`

PART 3 · RQ3 NOTE: WHY UNCERTAINTY IS HARD TO QUANTIFY (JUNE 2023 RECORD)

RQ3: why the uncertainty is hard to quantify (response to feedback)

> **Update (year split):** the plan below has been carried out on the 2023 test year. RQ3 now has a two-part wording, (a) does σ̂ point at the hard pixels and (b) a guaranteed 90% range, calibrated on 2022. See `rq3_uncertainty_results.md`. This note is kept as the June 2023 record.

Feedback received: "*I like the idea and where the report is going, but the uncertainty may be hard to quantify. Be careful with that.*"

Short answer: we agree, and our first results back this up. On average the confidence map looks well calibrated, but that average hides three problems. The coverage number shifts with the model and with the training epoch while accuracy barely changes. The map is overconfident exactly where it matters most. And our validation data is too narrow to tell any of this apart reliably. We therefore plan to narrow RQ3 to a calibration check with clearly stated limits, rather than claim the confidence map measures "the uncertainty" of sharpened AOD.

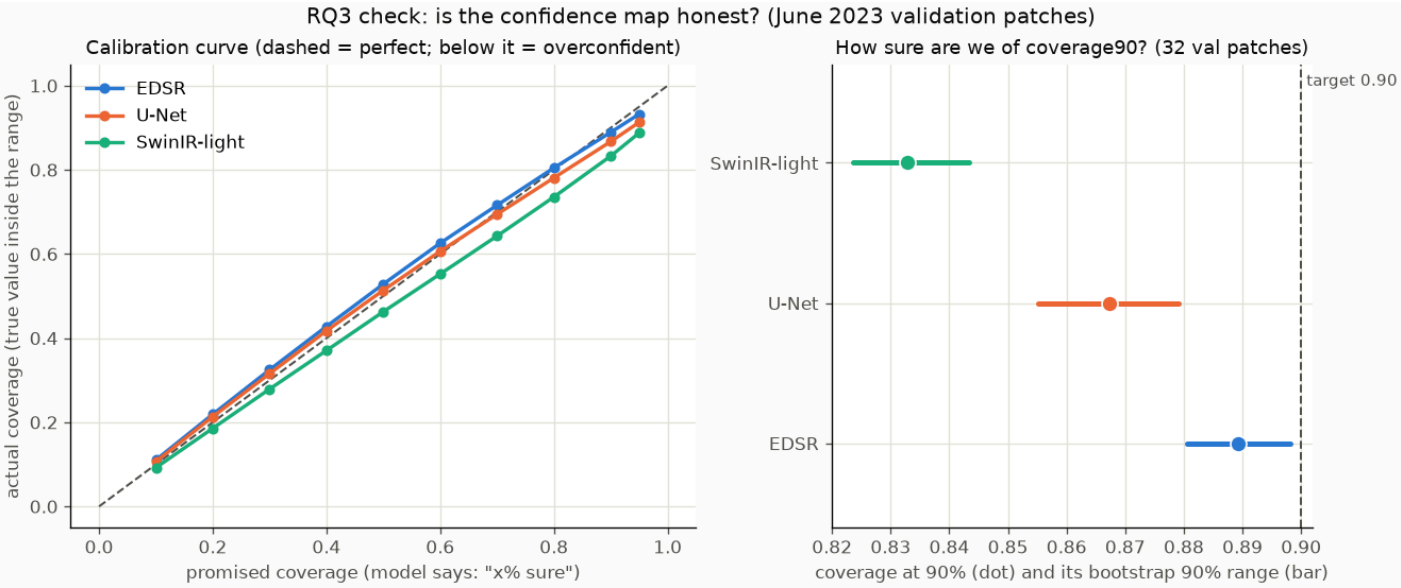
All numbers below come from the temporary June 2023 split: 112 training patches from 13 days, and 32 validation patches from **June 28–30**. Reproduce with:

```
python -W ignore -m src.rq3_uncertainty_check
```

What the confidence map is

Each model outputs a sharpened 1 km AOD map ŷ plus a per-pixel variance σ^2, trained with a Gaussian negative log-likelihood (Kendall & Gal, 2017). We check it with **coverage**: if the model says "90% sure", the true 1 km value should fall within ±1.645 σ̂ about 90% of the time.

Evidence 1: on average, calibration looks fine



Model	Coverage at 90%	Calibration error (10 levels)	Mean predicted σ	RMSE
EDSR	0.889	0.019	0.0128	0.0142
U-Net	0.867	0.017	0.0123	0.0142
SwinIR-light	0.833	0.041	0.0112	0.0140

Taken alone, this would suggest RQ3 is mostly solved. The next three points show why it isn't.

Evidence 2: the coverage number is unstable, while accuracy is stable

Within a **single training run**, coverage90 swings far more between epochs than RMSE does (epochs 100–300, scored every 25 epochs):

Model	RMSE range	Coverage90 range
EDSR	0.0140–0.0143	**0.85–0.95**
U-Net	0.0142–0.0150	**0.87–0.97**
SwinIR-light	0.0139–0.0142	**0.83–0.91**

The three models also end at different coverage (0.83–0.89) with practically the same RMSE (0.0140–0.0142). So **the calibration result depends on when training stops and which architecture is used**, not just on how good the model is. Any single coverage number we report could have come out as "well calibrated" or "clearly overconfident" depending on these choices.

Evidence 3: overconfident where it matters

Near cloud edges. Coverage90 for pixels next to a missing (cloud) pixel, vs pixels away from gaps:

Model	Near cloud edges (11,131 px)	Open sky
EDSR	0.84	0.89
U-Net	0.82	0.87
SwinIR-light	0.81	0.84

Cloud edges are where the model has the least information. Those are the pixels where a user most needs an honest confidence value, and that's where it falls furthest short.

Most-confident pixels. We split pixels into thirds by predicted $\hat{\sigma}$:

Model	Most sure	Middle	Least sure
EDSR	0.88	0.88	0.91
U-Net	0.84	0.86	0.91
SwinIR-light	**0.77**	0.83	0.90

The "least sure" pixels are well covered. The **pixels the model is most confident about are the most overconfident**: 77% actual vs 90% promised for SwinIR. The rank correlation between $\hat{\sigma}$ and actual |error| is only **0.37–0.40**. The map points in the right direction, but only roughly.

Evidence 4: the validation data can't tell these apart

- A bootstrap over the 32 validation patches gives narrow ranges, e.g. EDSR 0.881–0.898. But **all 32 patches come from 3 consecutive days** (June 28–30), and neighbouring patches overlap by half. The pixels are far from independent: effectively we have **3 weather situations**, not 116,926 pixels. The real uncertainty on coverage is much larger than the bootstrap suggests.
- None of those days is a smoke or dust event, the cases where public-health users would rely on the confidence map most.

Evidence 5: what the confidence map does NOT measure

The model is trained to predict MAIAC's own 1 km AOD from its 4 km average. So $\hat{\sigma}$ only describes **how hard it is to recover the 1 km MAIAC value from the 4 km one** (mean $\hat{\sigma} \approx 0.011$–$0.013$, about the size of the RMSE). It does **not** include:

1. **The satellite retrieval's own error.** MAIAC 1 km AOD is itself an estimate, and its error vs ground truth (e.g. AERONET) never enters our training target.
2. **Model uncertainty under new conditions.** A single network's $\hat{\sigma}$ captures noise it saw in training. It doesn't capture "I've never seen a day like this", e.g. smoke plumes or another season.
3. **Downstream uncertainty.** Nothing here says how uncertain a PM2.5 estimate would be.

Presenting $\hat{\sigma}$ as "the uncertainty of the sharpened AOD" would therefore overstate what it is.

What we plan to do (proposed scope for RQ3)

1. **Reword RQ3** to something like: "*Is the model's predicted per-pixel variance calibrated against held-out 1 km MAIAC AOD, and where does it fail?*" No claim about total or physical uncertainty.
2. **Report calibration, not just one number:** a calibration curve and coverage at several levels on the **2023 test year**, with uncertainty ranges computed **by day** rather than by patch.
3. **Report where it fails:** cloud edges, most-confident pixels and high-AOD days, as above.
4. **Fix the stopping-point problem:** choose the checkpoint on the 2022 validation year, then optionally rescale $\hat{\sigma}$ with one factor fitted on 2022 (post-hoc calibration). Report test-year coverage before and after rescaling.
5. **Optional, if time allows:** a small deep ensemble (≈5 EDSRs with different seeds) to capture model uncertainty as well. Where the ensemble disagrees on smoke days would show point 2 of Evidence 5 directly.
6. **State the limits explicitly** in the report (Evidence 5).

Questions for office hours

1. Is a calibration-focused RQ3 (points 1–4) an acceptable scope, or would you like the retrieval error included too (e.g. via AERONET)?
2. Is per-day bootstrapping on the 2023 test year an adequate way to report how uncertain the coverage estimate is?
3. Is post-hoc rescaling acceptable, as long as it's fitted on the validation year only?
4. Is the deep ensemble worth the extra compute, or should we keep RQ3 small and spend the time on data (2018–2023)?

Caveat: these results come from one month and three validation days, so they illustrate the problem rather than settle it. The final RQ3 numbers will use the 2018–21 / 2022 / 2023 split.

Reference: Kendall, A. & Gal, Y. (2017). *What Uncertainties Do We Need in Bayesian Deep Learning for Computer Vision?* NeurIPS.

PART 4 · RQ3 PLAN (TO-DO LIST)

To-do on the better machine: proving the uncertainty map (RQ3)

Why

The model outputs two maps per patch: the sharp AOD **ŷ** and its uncertainty **σ̂** (`log_var` in `src/models/sr_net.py`). The loss (`uncertainty_loss`) only *teaches* σ̂. To show σ̂ can be trusted, test it on data the model never saw (the 2023 test year) against the true 1 km values.

So far it has only been checked on 3 June days (`docs/rq3_uncertainty_note.md`). Those checks found it overconfident at cloud edges and on its most-confident pixels, and unstable between epochs.

Step 0: Get set up

- ☐ `git pull` on `ryan-dev` (it includes the merge with main).
- ☐ Check that `data/split_data/` contains `train.npz` (2018–21, 4543 patches), `val.npz` (2022, 1142), `test.npz` (2023, 976) and `train_stats.npz`.
- ☐ Check that the GPU works: `python -c "import torch; print(torch.cuda.is_available())"`.

Step 1: Train on the split by year

- ☐ In `src/dataset.py`, add a loader for `data/split_data/`:
  - reuse `read_real_patches()` for train, val and test
  - normalise with `mean` / `std` from `train_stats.npz`
- ☐ In `src/train.py`, add a `--data years` option. Pick the epoch using **2022 val only**, then score **2023 test** once at the end.
- ☐ Train EDSR, U-Net and SwinIR, each with and without `--consistency`.
- ☐ Sanity check: the model should beat bicubic on 2023 test (RMSE/PSNR).

Step 2: Error map vs uncertainty map (images)

For a few 2023 test patches, plot side by side:

```
LR input | sharp ŷ | true 1 km y | |ŷ − y| error | σ̂ uncertainty
```

- ☐ Draw the error and σ̂ panels with the same colour scale, and show cloud pixels as white.
- ☐ The plotting code in `src/overfit_one_patch.py` already has this layout. Add the σ̂ panel.
- ☐ Pick examples: a clear day, a cloudy day, and a high-AOD (smoky) day.
- ☐ **Look for:** bright spots in σ̂ should match bright spots in the error map. A spot that's bright in the error map but dark in σ̂ is overconfident.

Step 3: Prove it with every pixel in 2023 test

Use `src/rq3_uncertainty_check.py`, which already computes coverage and calibration:

- ☐ **Calibration curve:** at confidence levels 10%…95%, check how often the true value falls inside $\hat{y} \pm z \cdot \hat{\sigma}$. Plot promised vs actual; the diagonal means honest. This is `docs/diagrams/rq3_calibration.png` redone on 2023.
- ☐ **Error grows with $\hat{\sigma}$:** group pixels by $\hat{\sigma}$ (low → high) and plot the average real error per group. It should rise. Also report the rank correlation (`spearman()` already exists).
- ☐ **Throw out the unsure pixels:** remove the highest-$\hat{\sigma}$ pixels first and recompute RMSE. It should drop faster than when removing random pixels.
- ☐ **Where it fails:** compare coverage at cloud edges vs open sky, and on high-AOD vs normal patches.

Step 4: Fix it if it's off

- ☐ Find one number k so that $k \cdot \hat{\sigma}$ gives 90% coverage on **2022 val**.
- ☐ Apply k to 2023 test and report coverage before and after.

Step 5 (if time): Ensemble

- ☐ Train 5 EDSRs with seeds 0–4.
- ☐ Where they disagree is model uncertainty ("never seen this before"), which $\hat{\sigma}$ alone can't capture. Show it on the smoky-day example.

Needs from Rency

- ☐ The patch files store only the year. To bootstrap by day and label examples by date or season, ask her to also save `date` and patch `(row, col)` in `src/Data extraction/Data extraction_make_patches.py`. Her script already parses the date from the filename.

Done when

- ☐ 5-panel image figure (Step 2)
- ☐ Calibration curve, error-vs-$\hat{\sigma}$ plot and remove-pixels plot on 2023 (Step 3)
- ☐ Coverage before and after the k fix (Step 4)
- ☐ One paragraph of limits: $\hat{\sigma}$ does **not** include MAIAC's own measurement error (see `rq3_uncertainty_note.md`, Evidence 5)